Multi-Channel Notification Orchestration Agent

REQUIREMENTS DOCUMENT

Executive Summary

The Multi-Channel Notification Orchestration Agent is a centralized, intelligent communication gateway designed for Data Alchemy to manage all internal and external notifications across email, SMS, WhatsApp, Slack, Twilio voice, push notifications and in-app messaging. This unified platform eliminates fragmented communication systems, provides intelligent routing, ensures delivery reliability, maintains comprehensive audit trails, and scales to handle millions of messages per day.

Business Drivers:

- Fragmented communication channels causing inconsistent user experiences
- No centralized visibility into notification delivery and engagement
- Redundant vendor integrations across multiple applications
- Compliance requirements for communication audit trails
- Need for intelligent failover and delivery optimization

Key Capabilities:

- Unified API for all notification channels with consistent interface
- Intelligent routing based on user preferences, urgency, and availability
- Template management with multi-language and personalization support
- Delivery tracking with real-time status updates and analytics
- Automatic failover between channels for critical notifications
- Rate limiting, cost optimization, and vendor management
- Comprehensive audit logging for compliance (SOC 2, HIPAA, GDPR)

1. System Architecture

1.1 High-Level Architecture

The notification agent follows a layered, event-driven architecture with clear separation of concerns:

Architecture Layers:

1. API Gateway Layer: REST/GraphQL endpoints, authentication, rate limiting
2. Orchestration Layer: Message routing, priority queuing, channel selection
3. Provider Layer: Channel-specific adapters (Email, SMS, WhatsApp, etc.)
4. Persistence Layer: Message storage, status tracking, analytics

5. Monitoring Layer: Metrics, logging, alerting, tracing

Core Design Principles:

- Channel Abstraction: Applications use a single API regardless of delivery channel
- Provider Agnostic: Easy to swap or add new providers without code changes
- Async-First: Non-blocking operations with event-driven processing
- Idempotent: Safe to retry without duplicate sends
- Fail-Safe: Graceful degradation with automatic fallback

1.2 Technology Stack

Component	Technology	Purpose
Orchestration	AgentCore 2.0	Agent lifecycle management
State Machine	LangGraph	Conversation flow control
API Framework	FastAPI	RESTful API and WebSocket
LLM	Qwen 2.5/3, Mistral	Natural language generation
Embeddings	text-embedding-3-large	Vector representations
Vector DB	AWS OpenSearch	Semantic search
Database	Supabase PostgreSQL	Structured data
Cache	Redis 7.2	Session & rate limiting
Queue	AWS SQS/Kafka	Async job processing
Storage	AWS S3	Document storage
Monitoring	Cloudwatch	Observability
Time-Series DB	InfluxDB / TimescaleDB	Analytics and metrics

1.3 Component Diagram

The system consists of the following interconnected components:

Component	Responsibility
API Gateway	Authentication, validation, rate limiting, routing
Message Router	Channel selection, priority assignment, queuing
Template Engine	Render templates with personalization data
Provider Manager	Vendor abstraction, failover, load balancing
Delivery Tracker	Status updates, webhooks, retry logic
Analytics Engine	Metrics aggregation, reporting, insights
Audit Logger	Compliance logs, immutable event store
Scheduler	Delayed delivery, scheduled campaigns

2. Channel Specifications

2.1 Email (via AWS SES / Azure Graph API)

Capabilities:

- Transactional: Order confirmations, password resets, receipts
- Marketing: Newsletters, promotions, announcements
- Attachments: PDF invoices, reports (up to 10MB)
- HTML/Plain Text: Rich formatting with fallback
- Tracking: Opens, clicks, bounces, unsubscribes

SLA Targets:

- Delivery Time: < 5 seconds for transactional, < 30 min for bulk
- Success Rate: 99.5% (excluding hard bounces)
- Throughput: 10,000 emails/minute

Configuration:

Parameter	Value	Notes
Provider Priority	1. AWS SES, 2. Azure Graph API	Automatic failover
Retry Policy	3 attempts, exponential backoff	5s, 25s, 125s delays
Bounce Handling	Hard bounce = suppress future	Soft bounce = retry
Rate Limit	14 emails/second (SES)	Per-account limit

2.2 SMS (via Twilio)

Use Cases:

- OTP/2FA codes for authentication
- Transaction alerts and opportunity notifications
- Appointment reminders and confirmations
- Emergency alerts and system outages

Features:

- International: 200+ countries with local sender IDs
- Smart Routing: Optimize for cost and deliverability
- Alphanumeric Sender: Branded sender IDs (approval required)
- Delivery Reports: Real-time status via webhooks

Configuration:

Parameter	Value
Character Limit	160 (GSM-7), 70 (Unicode)
Cost Optimization	Route via cheapest provider per country
Delivery SLA	< 10 seconds
Validity Period	72 hours (carrier dependent)

2.3 WhatsApp Business (via Twilio)

Message Types:

- Template Messages: Pre-approved templates for notifications
- Session Messages: Free-form replies within 24-hour window
- Interactive: Buttons, lists, location sharing
- Media: Images, videos, PDFs, audio

Requirements:

- WhatsApp Business Account verified by Meta
- Message templates approved (usually 24-48 hours)
- Opt-in consent from recipients required
- Compliance with WhatsApp Commerce Policy

Rate Limits:

Tier	Messages/24hr	Criteria
Tier 1	1,000	New business accounts
Tier 2	10,000	After 7 days with < 50% block rate
Tier 3	100,000	Consistent quality score
Unlimited	No limit	Established, high-quality senders

2.4 Slack (via Slack API)

Capabilities:

- Channels: Public, private, and DM notifications
- Rich Formatting: Markdown, blocks, interactive components
- Mentions: @user, @channel, @here notifications
- Threads: Organized conversation context
- Reactions: Enable quick feedback

Integration Types:

- Webhooks: Simple incoming webhooks for basic notifications
- Bot User: Full API access with OAuth
- Slash Commands: Interactive commands from users

Best Practices:

- Use threads for related updates to reduce noise
- Implement user preferences for notification frequency
- Respect Do Not Disturb and working hours
- Use ephemeral messages for sensitive info

2.5 Push Notifications (via FCM, APNs)

Platforms:

- Android: Firebase Cloud Messaging (FCM)
- iOS: Apple Push Notification Service (APNs)
- Web: Progressive Web App push via FCM

Payload Structure:

Field	Description	Max Length
Title	Notification headline	65 chars (Android), 178 bytes (iOS)
Body	Message content	240 chars (Android), 4KB (iOS)
Icon	App icon or custom image	URL or local resource
Data	Custom key-value pairs	4KB total

Features:

- Silent Notifications: Update app state without alerting user
- Rich Media: Images, videos, action buttons
- Badge Count: Update app icon badge
- Sound & Vibration: Custom notification sounds
- Scheduling: Deliver at optimal time for user engagement

2.6 Voice Calls (via ElevenLabs)

Use Cases:

- Urgent alerts requiring immediate attention
- Two-factor authentication via voice OTP
- Appointment reminders with confirm/cancel options
- Emergency notifications (system outages, security)

Features:

- Text-to-Speech: Dynamic voice messages in 40+ languages
- IVR: Interactive voice response with keypad input
- Call Recording: Compliance and quality assurance
- Call Forwarding: Route to human agent if needed

2.7 In-App Notifications

Types:

- Toast/Snackbar: Temporary, non-intrusive alerts
- Banner: Persistent top/bottom screen notifications
- Inbox: Notification center with history
- Overlay: Modal dialogs for critical actions

Implementation:

- WebSocket: Real-time delivery for active sessions
- Polling: Fallback for unreliable connections
- Local Storage: Persist unread notifications

3. Core Features

3.1 Intelligent Routing

Priority-Based Routing:

6. CRITICAL: Immediate delivery via fastest channel (SMS, Voice, Push)
7. HIGH: Within 1 minute via preferred channel with fallback
8. MEDIUM: Within 5 minutes, optimize for cost
9. LOW: Within 1 hour, batch for cost efficiency

User Preference Routing:

- Respect user's preferred channels per notification type
- Honor Do Not Disturb schedules (e.g., 10 PM - 8 AM)
- Support opt-out from specific channels or categories

Automatic Failover:

- If primary channel fails (e.g., email bounces), try secondary
- If all channels fail, escalate to fallback (e.g., SMS after email+push fail)
- Configurable per notification type and urgency

3.2 Template Management

Template Features:

- Multi-Language: Support for 20+ languages with automatic locale detection
- Personalization: Variable substitution with fallback defaults
- Versioning: A/B testing and gradual rollout of new templates
- Preview: Render with sample data before deployment

Template Syntax:
Example Email Template:
Subject: {{user.first_name}}, your order #{{order.id}} is confirmed!
Body: Hi {{user.first_name}}, your order for {{order.total}} has been placed...

Supported Formats:

- HTML (Email): Responsive design with inline CSS
- Plain Text: Fallback for email, full content for SMS
- Markdown (Slack): Rich formatting for Slack messages
- JSON (WhatsApp): Structured template payload

3.3 Rate Limiting & Throttling

Global Limits:

Scope	Limit	Window
Per User	100 notifications/hour	Rolling 60 min

Per Application	10,000 notifications/minute	Fixed minute
Per Channel	Provider-specific	Varies
System-Wide	1 million notifications/minute	Auto-scaling

Smart Throttling:

- Distribute bulk sends evenly to avoid spikes
- Respect quiet hours per user timezone
- Batch similar notifications (e.g., daily digest)

3.4 Deduplication

Duplicate Prevention:

- Idempotency Key: Prevent duplicate sends from retries
- Content Hash: Detect identical messages sent within 5 minutes
- User Dedup: Max 1 notification per type per user per 10 minutes

Implementation:

- Redis Cache: Store message fingerprints with TTL
- Hash Algorithm: SHA-256 of (user_id + type + content)
- TTL: 24 hours for idempotency, 10 minutes for content dedup

3.5 Scheduled Delivery

Scheduling Options:

- Fixed Time: Deliver at specific UTC timestamp
- Optimal Time: ML-based best time for user engagement
- Recurring: Daily, weekly, monthly campaigns
- Conditional: Trigger after event (e.g., 24h after signup)

Timezone Handling:

- Store user timezone in profile
- Convert scheduled time to user's local time
- Handle DST transitions automatically

4. API Specifications

4.1 Send Notification (Synchronous)

Endpoint: POST /api/v1/notifications/send

Request Body:
{ "recipient": { "user_id": "user_12345", "email": "user@example.com", "phone": "+1234567890" },
"notification": { "type": "order_confirmation", "priority": "high", "channels": ["email", "push"], "subject": "Order Confirmed", "template_id": "tmpl_order_conf", "data": { "order_id": "ORD-9876", "total": "$129.99" } },
"options": { "idempotency_key": "ord_9876_conf", "track_opens": true, "track_clicks": true }}

Response:
{ "notification_id": "notif_abc123", "status": "queued", "channels": { "email": {"message_id": "msg_email_xyz", "status": "queued"}, "push": {"message_id": "msg_push_abc", "status": "sent"} }, "estimated_delivery": "2024-03-10T10:05:30Z"}

4.2 Batch Send (Async)

Endpoint: POST /api/v1/notifications/batch

Use Case: Send to thousands of users efficiently

Request:
{ "template_id": "tmpl_weekly_digest", "recipients": [{"user_id": "u1", "email": "u1@ex.com", "data": {...}}, {"user_id": "u2", "email": "u2@ex.com", "data": {...}}], "channel": "email", "schedule_at": "2024-03-10T15:00:00Z"}

Response:
{ "batch_id": "batch_789", "status": "processing", "total_recipients": 2, "estimated_completion": "2024-03-10T15:10:00Z"}

4.3 Get Status

Endpoint: GET /api/v1/notifications/{notification_id}

Response:
{ "notification_id": "notif_abc123", "status": "delivered", "channels": { "email": { "status": "delivered", "delivered_at": "2024-03-10T10:05:35Z", "opened": true, "clicked": false } }, "attempts": 1, "created_at": "2024-03-10T10:05:20Z"}

4.4 Webhooks

Event Types:

- notification.queued: Message accepted and queued
- notification.sent: Handed to provider successfully
- notification.delivered: Confirmed delivery to recipient
- notification.failed: Permanent failure (all retries exhausted)
- notification.bounced: Email hard bounce
- notification.opened: Email opened (tracking pixel)
- notification.clicked: Link clicked in email/push

Webhook Payload:

{ "event": "notification.delivered", "notification_id": "notif_abc123", "channel": "email", "timestamp": "2024-03-10T10:05:35Z", "metadata": {...}}

5. Database Schema

5.1 Core Tables

notifications

Column	Type	Description
id	UUID	Primary key
user_id	VARCHAR(50)	Recipient user ID
type	VARCHAR(50)	Notification type/category
priority	ENUM	critical|high|medium|low
status	ENUM	queued|sent|delivered|failed
template_id	VARCHAR(50)	Template used
data	JSONB	Template variables
scheduled_at	TIMESTAMP	When to send (NULL = immediate)
created_at	TIMESTAMP	Creation time
updated_at	TIMESTAMP	Last status update

notification_channels

Column	Type	Description
id	UUID	Primary key
notification_id	UUID	FK to notifications

channel	VARCHAR(20)	email|sms|whatsapp|slack|push
provider	VARCHAR(30)	aws_ses|twilio|fcm etc.
message_id	VARCHAR(100)	Provider message ID
status	ENUM	queued|sent|delivered|failed|bounced
attempts	INTEGER	Retry count
error_code	VARCHAR(50)	Provider error code
delivered_at	TIMESTAMP	Delivery confirmation time
opened_at	TIMESTAMP	Email opened time
clicked_at	TIMESTAMP	Link clicked time

templates

Column	Type	Description
id	VARCHAR(50)	Primary key (e.g., tmpl_order_conf)
name	VARCHAR(100)	Human-readable name
channel	VARCHAR(20)	email|sms|whatsapp etc.
language	VARCHAR(10)	en|es|zh etc.
subject	TEXT	Email subject or SMS preview
body	TEXT	Template content with {{variables}}
version	INTEGER	Version number for A/B testing
active	BOOLEAN	Is this template active
created_at	TIMESTAMP	Creation time

user_preferences

Column	Type	Description
user_id	VARCHAR(50)	Primary key

preferred_channels	JSONB	{type: [channels]} mapping
quiet_hours	JSONB	{start: '22:00', end: '08:00', timezone}
unsubscribed	TEXT[]	Array of notification types opted out
language	VARCHAR(10)	Preferred language
timezone	VARCHAR(50)	User timezone (IANA format)

6. Message Flow & Routing

6.1 End-to-End Flow

10. Application calls POST /api/v1/notifications/send
11. API Gateway authenticates, validates, applies rate limits
12. Router determines target channels based on priority and user prefs
13. Template Engine renders message with personalization data
14. Deduplication checks if identical message sent recently
15. Message published to appropriate queue (email-queue, sms-queue, etc.)
16. Channel Worker picks message from queue
17. Provider Adapter sends via vendor API (SES, Twilio, FCM, etc.)
18. Status updated in database (sent, delivered, failed)
19. Webhook sent to application with delivery status
20. Analytics aggregated for reporting dashboard

6.2 Routing Decision Logic

Pseudocode:

```
if priority == CRITICAL:
    channels = [SMS, Voice, Push]  # Fastest delivery
elif user_preferences.has_channel_for(notification_type):
    channels = user_preferences.get_channels(notification_type)
elif notification_type == 'marketing':
    channels = [Email]  # Cost-effective for bulk
elif notification_type == 'transactional':
    channels = [Email, Push]  # Reliable delivery
else:
    channels = [Email]  # Default fallback

# Apply quiet hours filter
if is_quiet_hours(user_timezone):
```

```
if priority < HIGH:
    schedule_for_later()
else:
    send_immediately()  # Override quiet hours for urgent
```

6.3 Retry Strategy

Attempt	Delay	Action
1st Failure	30 seconds	Retry same provider
2nd Failure	5 minutes	Retry same provider
3rd Failure	30 minutes	Switch to backup provider
All Failed	N/A	Mark as failed, trigger webhook, escalate

Exponential Backoff Formula:
delay = min(initial_delay * (2 ** attempt), max_delay)

7. Security & Compliance

7.1 Authentication & Authorization

API Authentication:

- API Keys: Fixed keys for server-to-server (rotation every 90 days)
- JWT Tokens: Short-lived tokens for user-initiated notifications
- OAuth 2.0: For third-party integrations (Slack, WhatsApp)

Authorization:

- RBAC: Role-based access (admin, developer, viewer)
- Scoping: Limit API key to specific channels or notification types
- Audit: Log all API calls with requester identity

7.2 Data Privacy

PII Protection:

- Encryption at Rest: AES-256 for database and S3
- Encryption in Transit: TLS 1.3 for all API calls
- Data Masking: Hide sensitive fields in logs (email → e***@example.com)
- Retention: Delete notification content after 90 days

GDPR Compliance:

- Right to Access: API to retrieve user's notification history
- Right to Erasure: Permanently delete user data on request
- Data Minimization: Only store necessary fields
- Consent Management: Track opt-ins for marketing notifications

7.3 Audit Logging

Events Logged:

- API Access: Who sent notification, when, and what data
- Status Changes: Every status transition with timestamp
- Configuration Changes: Template updates, preference changes
- Security Events: Failed auth, rate limit exceeded, suspicious activity

Log Retention:

- Hot Storage (Elasticsearch): Last 30 days for quick search
- Warm Storage (S3): 31-365 days for compliance
- Cold Archive (Glacier): 1-7 years for legal requirements